Python Básico

III. Operadores

Operadores relacionais

Guilherme Ferreira Silveira

gfsilveira@gmail.com

9 de abril de 2026

T7-E08

Conclusões – Operadores Aritméticos

- Operadores aritméticos
- Operadores atribuição
- Precedência de Operadores
- Operadores em objetos compostos nativos
- Operadores em objetos compostos módulos

Operadores Atribuição

Operador	Descrição	Código	Código
=	Recebe	objeto = valor	obj_1 = valor
+=	Soma	result = result + obj_1	result += obj_1
-=	Subtração	result = result - obj_1	result -= obj_1
*=	Multiplicação	result = result * obj_1	result *= obj_1
/=	Divisão	result = result / obj_1	result /= obj_1
//=	Inteiro da divisão	result = result // obj_1	result //= obj_1
%=	Resto da divisão	result = result % obj_1	result %= obj_1
**=	Exponencial	result = result ** obj_1	result **= obj_1

Operadores Relacionais

Operador	Descrição	Resultado	Código
==	Igualdade	False para 5 igual de 2	5 == 2 -> False
!=	Diferença	True para 5 diferente de 2	5 != 2 -> True
>	Maior	True para 5 maior que 2	5 > 2 -> True
>=	Maior ou igual	True para 5 maior ou igual a 2	5 >= 2 -> True
<	Menor	False para 5 menor que 2	5 < 2 -> True
<=	Menor ou igual	False para 5 menor ou igual a 2	5 <= 2 -> False
<= <	Entre	True para 2 menor ou igual a 3 e 3 menor que 5.	2 <= 3 < 5 -> True

Precedência de Operadores

Operador
( )
**
* / // %
+ -
< <= => >
== !=

Operadores em objetos simples

Python

int _ int

```python
a = 2
b = 3
c = 4
print(f"{a} == {b}: {a == b}")
print(f"{a} != {b}: {a != b}")
print(f"{a} > {b}: {a > b}")
print(f"{a} >= {b}: {a > b}")
print(f"{a} < {b}: {a < b}")
print(f"{a} <= {b}: {a < b}")
print(f"{a} <= {b} < {c}: {a <= b < c}")
```

[6] ✓ 0.0s

```
2 == 3: False
2 != 3: True
2 > 3: False
2 >= 3: False
2 < 3: True
2 <= 3: True
2 <= 3 < 4: True
```

int _ float

```python
a = 2
b = 3.5
c = 4
print(f"{a} == {b}: {a == b}")
print(f"{a} != {b}: {a != b}")
print(f"{a} > {b}: {a > b}")
print(f"{a} >= {b}: {a > b}")
print(f"{a} < {b}: {a < b}")
print(f"{a} <= {b}: {a < b}")
print(f"{a} <= {b} < {c}: {a <= b < c}")
```

[7] ✓ 0.0s

```
2 == 3.5: False
2 != 3.5: True
2 > 3.5: False
2 >= 3.5: False
2 < 3.5: True
2 <= 3.5: True
2 <= 3.5 < 4: True
```

int _ bool

```python
a = 1
b = False
c = 4
print(f"{a} == {b}: {a == b}")
print(f"{a} != {b}: {a != b}")
print(f"{a} > {b}: {a > b}")
print(f"{a} >= {b}: {a > b}")
print(f"{a} < {b}: {a < b}")
print(f"{a} <= {b}: {a < b}")
print(f"{a} <= {b} < {c}: {a <= b < c}")
```

[9] ✓ 0.0s

```
1 == False: False
1 != False: True
1 > False: True
1 >= False: True
1 < False: False
1 <= False: False
1 <= False < 4: False
```

Operadores em Tuplas

Python

```python
obj_tupla_int = (1, 0, 1, 0)
obj_tupla_bool = (True, False, True, False)
print(f"{obj_tupla_int} == {obj_tupla_bool}: {obj_tupla_int == obj_tupla_bool}")
print(f"{obj_tupla_int} != {obj_tupla_bool}: {obj_tupla_int != obj_tupla_bool}")
print(f"{obj_tupla_int} > {obj_tupla_bool}: {obj_tupla_int > obj_tupla_bool}")
print(f"{obj_tupla_int} >= {obj_tupla_bool}: {obj_tupla_int > obj_tupla_bool}")
print(f"{obj_tupla_int} < {obj_tupla_bool}: {obj_tupla_int < obj_tupla_bool}")
print(f"{obj_tupla_int} <= {obj_tupla_bool}: {obj_tupla_int < obj_tupla_bool}")
```

[10] ✓ 0.0s

```
(1, 0, 1, 0) == (True, False, True, False): True
(1, 0, 1, 0) != (True, False, True, False): False
(1, 0, 1, 0) > (True, False, True, False): False
(1, 0, 1, 0) >= (True, False, True, False): False
(1, 0, 1, 0) < (True, False, True, False): False
(1, 0, 1, 0) <= (True, False, True, False): False
```

Operadores em Tuplas/Listas

Python

```python
obj_tupla_int = (1, 0, 1, 0)
obj_list_bool = [True, False, True, False]
print(f"{obj_tupla_int} == {obj_list_bool}: {obj_tupla_int == obj_list_bool}")
print(f"{obj_tupla_int} != {obj_list_bool}: {obj_tupla_int != obj_list_bool}")
print(f"{obj_tupla_int} > {obj_list_bool}: {obj_tupla_int > obj_list_bool}")
print(f"{obj_tupla_int} >= {obj_list_bool}: {obj_tupla_int > obj_list_bool}")
print(f"{obj_tupla_int} < {obj_list_bool}: {obj_tupla_int < obj_list_bool}")
print(f"{obj_tupla_int} <= {obj_list_bool}: {obj_tupla_int < obj_list_bool}")
```

[22] 0.0s

```
(1, 0, 1, 0) == [True, False, True, False]: False
(1, 0, 1, 0) != [True, False, True, False]: True
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[22], line 5
      3 print(f"{obj_tupla_int} == {obj_list_bool}: {obj_tupla_int == obj_list_bool}")
      4 print(f"{obj_tupla_int} != {obj_list_bool}: {obj_tupla_int != obj_list_bool}")
----> 5 print(f"{obj_tupla_int} > {obj_list_bool}: {obj_tupla_int > obj_list_bool}")
      6 print(f"{obj_tupla_int} >= {obj_list_bool}: {obj_tupla_int > obj_list_bool}")
      7 print(f"{obj_tupla_int} < {obj_list_bool}: {obj_tupla_int < obj_list_bool}")

TypeError: '>' not supported between instances of 'tuple' and 'list'
```

Operadores em Listas

Python

```python
obj_list_int = [1, 0, 1, 0]
obj_list_bool = [True, False, True, False]
print(f"{obj_list_int} == {obj_list_bool}: {obj_list_int == obj_list_bool}")
print(f"{obj_list_int} != {obj_list_bool}: {obj_list_int != obj_list_bool}")
print(f"{obj_list_int} > {obj_list_bool}: {obj_list_int > obj_list_bool}")
print(f"{obj_list_int} >= {obj_list_bool}: {obj_list_int > obj_list_bool}")
print(f"{obj_list_int} < {obj_list_bool}: {obj_list_int < obj_list_bool}")
print(f"{obj_list_int} <= {obj_list_bool}: {obj_list_int < obj_list_bool}")
```

[23] ✓ 0.0s

```
[1, 0, 1, 0] == [True, False, True, False]: True
[1, 0, 1, 0] != [True, False, True, False]: False
[1, 0, 1, 0] > [True, False, True, False]: False
[1, 0, 1, 0] >= [True, False, True, False]: False
[1, 0, 1, 0] < [True, False, True, False]: False
[1, 0, 1, 0] <= [True, False, True, False]: False
```

Operadores em Dicionários

Python

```python
obj_dict_int = {0: 1, 1: 0, 2: 1, 3: 0}
obj_dict_bool = {0: True, 1: False, 2: True, 3: False}
print(f"{obj_dict_int} == {obj_dict_bool}: {obj_dict_int == obj_dict_bool}")
print(f"{obj_dict_int} != {obj_dict_bool}: {obj_dict_int != obj_dict_bool}")
print(f"{obj_dict_int} > {obj_dict_bool}: {obj_dict_int > obj_dict_bool}")
print(f"{obj_dict_int} >= {obj_dict_bool}: {obj_dict_int > obj_dict_bool}")
print(f"{obj_dict_int} < {obj_dict_bool}: {obj_dict_int < obj_dict_bool}")
print(f"{obj_dict_int} <= {obj_dict_bool}: {obj_dict_int < obj_dict_bool}")
```

[24] 0.0s

```
{0: 1, 1: 0, 2: 1, 3: 0} == {0: True, 1: False, 2: True, 3: False}: True
{0: 1, 1: 0, 2: 1, 3: 0} != {0: True, 1: False, 2: True, 3: False}: False
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[24], line 5
      3 print(f"{obj_dict_int} == {obj_dict_bool}: {obj_dict_int == obj_dict_bool}")
      4 print(f"{obj_dict_int} != {obj_dict_bool}: {obj_dict_int != obj_dict_bool}")
----> 5 print(f"{obj_dict_int} > {obj_dict_bool}: {obj_dict_int > obj_dict_bool}")
      6 print(f"{obj_dict_int} >= {obj_dict_bool}: {obj_dict_int > obj_dict_bool}")
      7 print(f"{obj_dict_int} < {obj_dict_bool}: {obj_dict_int < obj_dict_bool}")

TypeError: '>' not supported between instances of 'dict' and 'dict'
```

Operadores em Arrays

Python

```python
obj_array_int = np.array([1, 0, 1, 0])
obj_array_bool = np.array([True, False, True, False])
print(f"{obj_array_int} == {obj_array_bool}: {obj_array_int == obj_array_bool}")
print(f"{obj_array_int} != {obj_array_bool}: {obj_array_int != obj_array_bool}")
print(f"{obj_array_int} > {obj_array_bool}: {obj_array_int > obj_array_bool}")
print(f"{obj_array_int} >= {obj_array_bool}: {obj_array_int > obj_array_bool}")
print(f"{obj_array_int} < {obj_array_bool}: {obj_array_int < obj_array_bool}")
print(f"{obj_array_int} <= {obj_array_bool}: {obj_array_int < obj_array_bool}")
```

[25] ✓ 0.0s

```
[1 0 1 0] == [ True False  True False]: [ True  True  True  True]
[1 0 1 0] != [ True False  True False]: [False False False False]
[1 0 1 0] > [ True False  True False]: [False False False False]
[1 0 1 0] >= [ True False  True False]: [False False False False]
[1 0 1 0] < [ True False  True False]: [False False False False]
[1 0 1 0] <= [ True False  True False]: [False False False False]
```

Operadores em DataFrames

```python
obj_pandas_int = pd.Series([1, 0, 1, 0])
obj_pandas_bool = pd.Series([True, False, True, False])

print("-" * 40)
print(f"{obj_pandas_int == obj_pandas_bool}")
print("-" * 40)
print(f"{obj_pandas_int != obj_pandas_bool}")
print("-" * 40)
print(f"{obj_pandas_int > obj_pandas_bool}")
print("-" * 40)
print(f"{obj_pandas_int > obj_pandas_bool}")
print("-" * 40)
print(f"{obj_pandas_int < obj_pandas_bool}")
print("-" * 40)
print(f"{obj_pandas_int < obj_pandas_bool}")
```

[29] ✓ 0.0s

```
----------------------------------------
0    True
1    True
2    True
3    True
dtype: bool
----------------------------------------
0    False
1    False
2    False
3    False
dtype: bool
----------------------------------------
```

Conclusões

- Operadores relacionais
- Operadores relacionais != atribuição
- Precedência de Operadores
- Operadores em objetos compostos nativos
- Operadores em objetos compostos módulos

Vamos praticar?
LIMC-IA

Vamos exercitar?
LIMC-IA

Exercícios

01 – Obtenha os resultados e retorne o código Python dos seguintes cálculos:

a = 2.0 + 3.0 / 5.0 == 10

b = (a + 10.2) - 5.0 * 2.0 <= 2.9

c = 10 <= a / b + 33.8 < 100

d = 5 != a / (b + c)

e = 33 == d * 2

Exercícios

02 – Altere os operadores relacionais para modificar o resultado de cada uma das comparações.

```
(1, 0, 1, 0) == (True, False, True, False): True
(1, 0, 1, 0) != (True, False, True, False): False
```

('a',1,True,'b',0,False) != ('a',True,1,'b',False,0)

['a',1,True,'b',0,False] != ('a',True,1,'b',False,0)

['a',1,True,'b',0,False] != ['a',True,1,'b',False,0]

['a',1,True,'b',0,False] > ['a',True,1]

['a',1,True,'b',0,False] == 1

['a',True,1] <= ['a',True,1] < ['a',1,True,'b',0,False]

Exercícios

03 – Usando o método DataFrame.to_csv('nome_do_arquivo.csv') exporte o objeto obtido no exercício 10 da aula anterior para um arquivo CSV. Usando o método pd.read_csv('nome_do_arquivo.csv') instancie uma DataFrame. Use o valor 'Unnamed: 0' no parâmetro index_col, para ajustar o index. Arredonde os valores para até três casas decimais, usando o método round().

Recupere a DataFrame.

Exercícios

04 – Instancie um numpy array que apresente o mesmo formato (linhas e colunas) da DataFrame. Este array deve conter apenas inteiros uns.

Recupere o array.

Obs.: Procure na internet o método numpy.ones()

Exercícios

05 – Quantos dos valores contidos na DataFrame são iguais aos valores no array?

06 – Quantos dos valores contidos na DataFrame são superiores aos valores no array?

07 – Quantos dos valores contidos na DataFrame são iguais ou inferiores aos valores no array?

08 – Quantos dos valores contidos na DataFrame estão entre 0 e 1?

Exercícios

09 – Quantos dos valores das médias das linhas são inferiores ou iguais à 5?

Obs 1.: Usando o método de classe, você pode converter qualquer objeto em outra classe. Ex. obj = list(array)

Obs 2.: Assim com o objeto da classe str, listas também possuem a função count()

Exercícios

10 – Quantos dos valores das médias das colunas são superiores à 5?

Python Básico

9 de abril de 2026